Evaluating a Structure-Aware Language Model for S1000D XML Document Retrieval

Cindy Y.S. Hung
International College of Innovation
National Chengchi University
Taiwan
yunsin.hung@gmail.com

Jr-Ping Wang
Department of Naval Architecture and Ocean Engineering
National Kaohsiung University of Science and Technology, Taiwan
jpw@nkust.edu.tw

Owen H.T. Lu*
International College of Innovation
National Chengchi University
Taiwan
owen.lu.academic@gmail.com

***Abstract*—S1000D uses XML to organize technical publications as structured data modules. Content is held not only in text, but also within tags, nesting and references. Learning representations for retrieving these documents is an important application of structure-aware language models. This work evaluates SANTA for S1000D XML document retrieval. We compare pooling methods, similarity functions, input lengths, and pre-training with Structured Data Alignment (SDA) and Masked Entity Prediction (MEP). We test 21 queries against pools of 21 and 332 documents, using both original and anonymized queries. SANTA's first-token representation performs close to random ranking, with an MRR of about 0.17 on original queries. Mean-pooled encoder outputs with cosine similarity raise the pre-trained model to 0.917, averaged across three seeds and pre-training sources. In our single-seed ablation study, raising the document limit from 256 to 1024 tokens increases the score from 0.560 to 0.927. SDA/MEP pre-training does not show consistent improvement, and its benefits vary with the data source, training duration, and query formulation. BM25 ranks nearly every relevant document first among 21 candidates, but with 332 candidates the pre-trained dense model reaches an MRR@100 of 0.658 against BM25's full-ranking MRR of 0.125. These results show that representation choices and retrieval settings strongly affect SANTA's performance on XML documents, while the benefits of structure-aware pre-training depend on the application conditions.**

***Index Terms*—dense retrieval, S1000D, XML, structure-aware pre-training**

I. Introduction

S1000D is an international specification maintained by the Aerospace, Security and Defense Industries Association of Europe (ASD), Aerospace Industries Association of America (AIA), and ATA e-Business Program for technical publications in aerospace, maritime, and other manufacturing industries. S1000D uses XML documents to organize technical content into reusable data modules (DMs) [1]. Much of that content's meaning comes from the tags, attributes, nested elements, and references in the XML that gives each passage a context beyond its wording. For example, similar descriptions may appear under different procedures or applicability conditions. A retrieval model therefore needs to match a query with the relevant content while accounting for the context expressed by the document structure. Simply retaining XML markup in the input does not establish whether the model uses that information effectively.

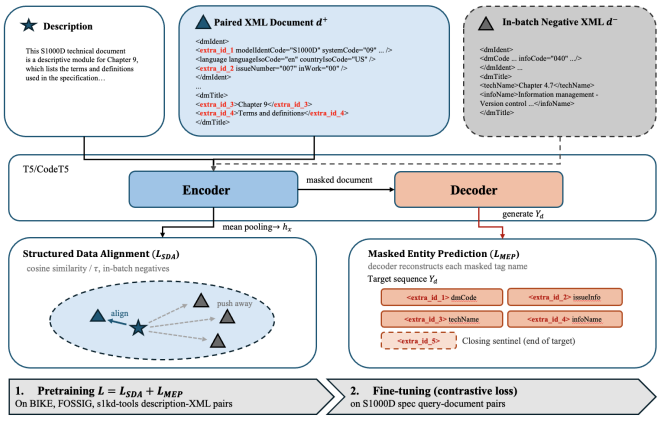


Fig. 1. SANTA-based XML retrieval: SDA/MEP pre-training followed by fine-tuning on S1000D query–document pairs.

Recent S1000D studies address document preparation and information access. Automated wiring-list preparation reduces manual effort and content errors [2], while interactive wiring manuals connect diagrams with related technical information [3]. SALERNO prepares documents for downstream machine learning while retaining applicability conditions and references [4], and an AND/OR tree model filters content for specific configurations and user contexts [5]. These studies demonstrate the practical value of structural relationships. However, their rule-based conversion, linking, and filtering methods do not directly evaluate learned representations for semantic and structural retrieval from S1000D XML documents.

This work adopts SANTA [6] as a baseline for XML retrieval evaluation. Figure 1 illustrates the training process. SANTA is pre-trained to align structured data with natural-language descriptions and to reconstruct masked entities. Its retrieval objective makes it a relevant starting point, while its reported performance on code retrieval motivates evaluation in a different structured data domain. The question is whether these learned representations can support effective retrieval

*Corresponding author: Owen H.T. Lu (owen.lu.academic@gmail.com).

from S1000D XML documents, whose tags, hierarchy, and references differ from those in code. Our choice of SANTA is therefore based on its training objectives and task relevance; its applicability to S1000D remains an empirical question.

This work examines the conditions under which SANTA-based representations support retrieval from S1000D XML documents. We evaluate retrieval using mean reciprocal rank (MRR), compare representation and pre-training choices, and analyze how query formulation and candidate pool size affect the results. These preliminary results provide evidence for adapting structure-aware language models to XML retrieval and inform later work on structured data generation. The experiments assess retrieval behavior; they do not directly establish preservation of individual XML relationships or the ability to generate valid modules. Our contributions are as follows:

1) An analysis of representation choices shows that SANTA's first-token representation performs near random ranking, while mean-pooled encoder outputs with cosine similarity and a 1024-token input limit support effective S1000D retrieval.
2) An evaluation of SDA/MEP pre-training across three external corpora, same-domain S1000D data, and two training durations finds no consistent benefit over fine-tuning alone.
3) A comparison of dense retrieval and BM25 using original and anonymized queries shows that their relative performance depends on candidate pool size.

The remainder of this paper is organized as follows. Section II reviews S1000D document processing and approaches to structural representation learning. Section III presents our methodology, and Section IV describes the dataset, evaluation metrics, and parameter settings. Section V reports retrieval performance, embedding visualizations, and a case study. Section VI summarizes findings and future directions.

II. Related Work

Existing S1000D applications preserve certain structural semantics through explicit processing rules. SALERNO groups extracted text by applicability, resolves references, and retains document relationships in a simple format for subsequent machine learning [4]. The AND/OR tree approach evaluates nested conditions to identify which documents or sections apply to a given context [5]. For document preparation, Zhang combines XML field extraction, source selection, configuration reconstruction, and schema mapping to generate wiring-list modules [2]. Based on three years of use for a large passenger aircraft, the author reports savings of 16,300 labor hours during initial preparation, alongside reductions in revision time and content errors. Cui et al. connect wiring diagrams, equipment information, and other manuals through data modules, graphical hotspots, and indexes [3]. These approaches show the importance of structure in operational processes. Conditions may determine applicability, and identifiers can connect related content. Their contribution focuses on data processing, rule-based filtering and document use, rather than learning structural representations for query–document matching.

Structural representation methods offer several ways to bring such information into machine learning. Vercoustre et al. use paths and subpaths combined with text to represent XMLs for document clustering [7]. Tree-LSTM combines child-node information to learn hierarchical representations, whilst evaluating sentiment classification and sentence relatedness [8]. Woof and Chen consider element paths, data types, and container properties in a framework for semantic trees [9]. Their main experiments use JSON representations, while an XML extension describes how attributes and ordered child nodes can be encoded. R-GCNs address a different aspect of structure by aggregating information according to relation type and direction in knowledge graphs [10]. Every method captures a different part of the structure. Paths describe context position, recursive composition models hierarchy, and typed edges represent explicit relations. None of these methods, however, has been tested on S1000D retrieval. Structure-aware language models incorporate these ideas into text encoding. MarkupLM represents tag paths and node positions through XPath embeddings and learns parent, child, sibling, and ancestor relations through pre-training [11]. DOM-LM combines tags, attributes, and tree positions, and preserves surrounding structural context when splitting extensive HTML documents [12]. WebFormer connects HTML nodes through graph attention and links each node to its text through a separate attention mechanism [13]. Their evaluations on webpage extraction and question answering show how giving structural information to models can improve document understanding.

However, the use for S1000D document retrieval would require decisions about XML adaptation, representation for documents, and query alignment. These studies also distinguish keeping markup in an input sequence from explicitly encoding paths or using relationships to control information processing. SANTA approaches structure-aware learning through retrieval-oriented pre-training objectives. Structured Data Alignment (SDA) contrastively aligns structured data with natural-language descriptions, while Masked Entity Prediction (MEP) reconstructs masked entities from context [6]. It uses the T5-family encoder–decoder architecture [14], with CodeT5-base as the base model for code retrieval. The original study performs continued pre-training on code–documentation pairs from CodeSearchNet [15] and downstream fine-tuning on the Adv dataset from CodeXGLUE [16]. SANTA reaches an MRR@100 of 0.461 without downstream fine-tuning and 0.475 after fine-tuning with hard negatives, compared with 0.167 for fine-tuned BERT and 0.272 for fine-tuned CodeBERT [6]. Unlike the models above, SANTA learns structure through alignment and entity reconstruction rather than by explicitly modeling XML paths or references, which makes it an appropriate baseline for exploring whether representations learnt through these pre-training tasks transfer to S1000D XML retrieval. Because SANTA also has a decoder trained to reconstruct masked content, it could later be extended to generating valid S1000D modules. Generation would require

separate training and metrics to evaluate schema validity, structure completeness and content accuracy.

III. Methodology

Dense retrieval represents queries and documents as vectors and ranks documents by their similarity to the query. Figure 1 outlines the training process. We follow SANTA and continue pre-training a T5-family encoder–decoder on XML documents d paired with natural-language descriptions u before fine-tuning on queries q. Each input $x \in \{q, u\}$ is encoded as a vector h_x. SANTA uses the final decoder layer's hidden state at the first decoding step. In our preliminary runs on S1000D data, this representation ranked documents close to random, so we also evaluate mean pooling. We average the final-layer encoder outputs $e_1, \ldots, e_n$ over the n non-padding tokens,

$$h_x = \frac{1}{n} \sum_{i=1}^{n} e_i. \tag{1}$$

so that every token of the input contributes to its representation. Relevance is then scored as

$$f(x, d) = \frac{\mathrm{sim}(h_x, h_d)}{\tau}, \tag{2}$$

where sim is the dot product or cosine similarity and τ is a temperature. SDA pulls each description toward its paired document d^+ and away from the other documents in the batch, D^-:

$$L_{\mathrm{SDA}} = -\log \frac{e^{f(u,d^+)}}{e^{f(u,d^+)} + \sum_{d^- \in D^-} e^{f(u,d^-)}}. \tag{3}$$

For MEP, we treat XML tags as the entities to be masked. Selected tags in d are replaced with sentinel tokens $\langle \mathrm{mask} \rangle_i$ to form X_d^{mask}, and the decoder reconstructs the target sequence Y_d, which pairs each sentinel with its original tag, token by token:

$$L_{\mathrm{MEP}} = -\sum_{j=1}^{k} \log P\big(t_j \mid X_d^{\mathrm{mask}}, t_{1:j-1}\big), \tag{4}$$

where $Y_d = (t_1, \ldots, t_k)$.

IV. Experimental Methodology

A. Dataset

We use XML documents paired with natural-language descriptions from four corpora (Table I). BIKE, FOSSIG, and s1kd-tools are used separately for pre-training, followed by fine-tuning on S1000D spec. For same-domain pre-training, we split the S1000D spec training set into two halves, pre-train on one and fine-tune on the other, and test both directions against fine-tuning alone. We also evaluate an alternate split, keeping the development and test sets unchanged.

Evaluation uses 21 test queries, each with one relevant document, against either 21 or 332 candidate documents. Original queries may contain document identifiers; anonymized queries are regenerated without them.

TABLE I
Query–document pairs by corpus and split; half-set experiments share the S1000D spec development and test sets.

Corpus	Role	Train	Dev	Test
BIKE	Pre-train	109	3	6
FOSSIG	Pre-train	8	2	2
s1kd-tools	Pre-train	39	4	3
S1000D spec	Fine-tune / test	297	14	21
S1000D half A / B	Same-domain	150 / 147	14	21
S1000D half A / B (alt)	Same-domain	143 / 154	14	21

B. Evaluation

We use MRR to evaluate retrieval performance, following the code retrieval evaluation in SANTA [6]. MRR is also used in code search studies such as CodeXGLUE [16] and CodeBERT [17], although their evaluation settings are not necessarily identical. The metric measures how highly the first relevant document is ranked for each query. Let Q be the set of evaluation queries and let r_q be the one-based rank of the first relevant XML document for query q. We compute

$$\mathrm{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{r_q}. \tag{5}$$

Each query has exactly one relevant document, given by the reference labels, and its score is the reciprocal of the rank at which that document is retrieved: 1 if it comes first, 0.5 if it comes second, and so on. We then average these scores over all queries, so MRR lies between 0 and 1, with higher values meaning the correct document tends to be ranked near the top. Given the small test corpora for S1000D spec (21 candidates), a random ranking gets a non-trivial score of about 0.17, so we report this chance level alongside each result. For the larger 332-document setting, dense models and the random baseline use MRR@100, with zero credit when the relevant document is outside the top 100. BM25 retains full-ranking MRR; this is an upper bound on its MRR@100.

C. Parameters

We initialize the models from the Hugging Face checkpoints t5-base and Salesforce/codet5-base. Training uses the PyTorch AdamW optimizer with default β values, $\epsilon = 10^{-8}$, zero weight decay, and gradient clipping at 1.0. The batch size is set to 16, with one gradient accumulation step. We use a linear learning-rate decay schedule with a warmup ratio of 0.1. SDA/MEP pre-training uses a learning rate of 5×10^{-5} for 10 epochs, with a 100-epoch setting in the ablation study. Fine-tuning uses a learning rate of 2×10^{-5} for 12 epochs.

The maximum lengths are 50 tokens for queries, 1024 tokens for documents, and 64 tokens for MEP target sequences. Configurations marked $p = 256$ limit documents to 256 tokens.

For contrastive training, we use in-batch negatives. Each query is paired with its matching document, and the other documents in the same batch serve as negatives. With a batch size of 16, this gives each query one positive and 15 negatives.

Mean-pooling configurations with cosine similarity use a temperature of 0.05, while first-token configurations use dot-product similarity with a temperature of 1. We also evaluate mean pooling with dot-product scoring in the ablation study.

For MEP, we randomly mask 50% of non-root XML tags from an S1000D schema whitelist.

We select the checkpoint with the highest development-set MRR. During pre-training, selection uses the SDA development pairs from the corresponding pre-training corpus. During fine-tuning, selection uses the S1000D spec development set.

V. Evaluation Results

We evaluate SANTA-based retrieval on the S1000D spec corpus described in Section IV-A and examine the effects of representation construction and pre-training. Unless stated otherwise, results use 21 test queries and 21 candidate documents. We also compare original and anonymized queries and expand the candidate pool to 332 documents. All reported retrieval scores are multiplied by 100, and differences are expressed in points. Cosine similarities are not rescaled. Dense retrieval uses MRR with 21 candidates and MRR@100 with 332 candidates. The 332-candidate BM25 results use full-ranking MRR, as noted in Table III.

A. Overall Performance

Table II reports trained-model performance averaged over three random seeds with 21 candidates. BM25 ranks the relevant document first for all 21 original queries (100.00) and for 20 anonymized queries (97.62). This strong result indicates that lexical matching is effective in a small candidate pool, even after explicit identifiers are removed.

SANTA's representation (using the decoder state at the first decoding step) gives weak retrieval performance for S1000D XMLs. Untrained T5 and CodeT5 score 17.33 and 17.39 on the original queries. After SDA/MEP pre-training and fine-tuning, the three first-token configurations average 17.01–17.87, close to the random baseline of 17.36. Their anonymized-query averages are 17.17–19.77. Section V-C relates this behavior to the geometry of the learned representations.

Adapting the representation to mean-pooled encoder outputs with cosine similarity substantially improves retrieval. In Table II, the pre-trained model averages 91.66 on original queries and 84.20 on anonymized queries, while the model without pre-training reaches 94.05 and 85.94. Anonymization reduces these scores by 7.46 and 8.11 points, respectively, compared with 2.38 points for BM25. These results motivate examining representation construction separately from the pre-training objectives.

a) Candidate Pool Size: Table III reports single-seed results for 21 and 332 candidates using the same queries and checkpoints within each row. For the 332-candidate pool, dense models and Random use MRR@100, while BM25 uses full-ranking MRR. With 332 candidates, BM25 falls to 12.50 on original queries and 11.47 on anonymized queries. The dense model without pre-training scores 63.78 and 45.17, while the pre-trained model averages 65.77 and 46.91. Across pre-training sources, scores range from 61.93 to 71.60 on original queries and from 38.72 to 51.46 on anonymized queries. As seen in Table III, dense retrieval exceedsBM25's MRR in the larger pool.

Relative to the 21-candidate results in Table III, BM25 loses 87.50 and 86.15 points. The dense model without pre-training loses 27.89 and 44.51 points, while the pre-trained average loses 26.13 and 39.58 points. At 332 candidates, pre-training improves the average over no pre-training by 1.99 points on original queries and 1.74 points on anonymized queries.

TABLE II
Retrieval with 21 candidates (MRR ×100). Trained-model results average three seeds. Avg. denotes the mean across pre-training sources.

Original queries				
Model	BIKE	FOSSIG	s1kd-tools	Avg.
Unsupervised				
Random	-	-	-	17.36
BM25	-	-	-	100.00
Zero-shot (first-token pooling)				
T5 (untrained)	-	-	-	17.33
CodeT5 (untrained)	-	-	-	17.39
Fine-tuning (first-token pooling, dot product)				
T5 + SDA/MEP (p=256)	16.71	20.25	16.66	17.87
CodeT5 + SDA/MEP (p=256)	17.77	16.31	16.96	17.01
CodeT5 + SDA/MEP (p=1024)	15.38	19.76	17.56	17.56
Fine-tuning (mean pooling, cosine)				
CodeT5 (no pre-training)	-	-	-	94.05
CodeT5 + SDA/MEP	85.03	95.24	94.71	91.66
Anonymized queries				
Model	BIKE	FOSSIG	s1kd-tools	Avg.
Unsupervised				
Random	-	-	-	17.36
BM25	-	-	-	97.62
Zero-shot (first-token pooling)				
T5 (untrained)	-	-	-	17.36
CodeT5 (untrained)	-	-	-	17.36
Fine-tuning (first-token pooling, dot product)				
T5 + SDA/MEP (p=256)	21.28	14.43	15.81	17.17
CodeT5 + SDA/MEP (p=256)	19.79	16.76	17.29	17.94
CodeT5 + SDA/MEP (p=1024)	27.01	14.10	18.20	19.77
Fine-tuning (mean pooling, cosine)				
CodeT5 (no pre-training)	-	-	-	85.94
CodeT5 + SDA/MEP	76.28	86.11	90.21	84.20

TABLE III
Candidate pool comparison (mean pooling, cosine, $p = 1024$; scores ×100). Average is across sources. At 332 candidates, BM25 uses full MRR; others use MRR@100.

	Original		Anonymized	
Model	21 docs	332 docs	21 docs	332 docs
Random	17.36	1.56	17.36	1.56
BM25	100.00	12.50	97.62	11.47
CodeT5, no pre-training	91.67	63.78	89.68	45.17
CodeT5 + SDA/MEP, pre-trained on				
BIKE	87.62	61.93	80.12	38.72
FOSSIG	92.86	63.78	86.51	51.46
s1kd-tools	95.24	71.60	92.86	50.55
Average	91.90	65.77	86.49	46.91

TABLE IV

SINGLE-SEED ABLATIONS. AVG. DENOTES THE MEAN ACROSS SOURCES; $p = 1024$ UNLESS SPECIFIED. EMBEDDING COMPARISONS USE 10-EPOCH SDA/MEP PRE-TRAINING.

Original queries				
Model	BIKE	FOSSIG	s1kd-tools	Avg.
Embedding (pre-trained with SDA + MEP)				
T5, first-token, dot (p = 256)	16.71	20.25	16.66	17.87
CodeT5, first-token, dot (p = 256)	17.77	16.31	16.96	17.01
CodeT5, first-token, dot	15.38	19.76	17.56	17.56
CodeT5, mean, dot	85.60	84.76	92.06	87.48
CodeT5, mean, cosine (p = 256)	53.85	59.23	54.81	55.97
CodeT5, mean, cosine	89.84	92.86	95.24	92.65
Pre-training (mean pooling, cosine)				
No pre-training	-	-	-	91.67
w/ SDA (no MEP)	83.33	92.86	89.68	88.62
w/ MEP (no SDA)	88.78	89.29	95.24	91.10
w/ SDA + MEP (10 epochs)	89.84	92.86	95.24	92.65
w/ SDA + MEP (100 epochs)	87.30	85.71	94.44	89.15
Anonymized queries				
Model	BIKE	FOSSIG	s1kd-tools	Avg.
Embedding (pre-trained with SDA + MEP)				
T5, first-token, dot (p = 256)	21.28	14.43	15.81	17.17
CodeT5, first-token, dot (p = 256)	19.79	16.76	17.29	17.94
CodeT5, first-token, dot	27.01	14.10	18.20	19.77
CodeT5, mean, dot	70.43	80.32	89.68	80.14
CodeT5, mean, cosine (p = 256)	38.72	27.49	23.77	29.99
CodeT5, mean, cosine	80.12	86.51	92.86	86.49
Pre-training (mean pooling, cosine)				
No pre-training	-	-	-	89.68
w/ SDA (no MEP)	73.97	83.73	90.48	82.72
w/ MEP (no SDA)	85.71	87.30	77.38	83.47
w/ SDA + MEP (10 epochs)	80.12	86.51	92.86	86.49
w/ SDA + MEP (100 epochs)	65.16	74.68	91.67	77.17

TABLE V

SAME-DOMAIN PRE-TRAINING (PT): MEAN POOLING, COSINE, $p = 1024$; MRR $\times 100$. TRAINING SPLITS: 150/147 (ORIGINAL) AND 143/154 (ALTERNATE).

Fine-tune on half B			
Split / seed	No PT	PT on A	Δ
Original queries			
Original, seed 42	86.31	89.88	+3.57
Original, seed 1	83.78	82.02	-1.75
Original, seed 2	79.93	84.07	+4.14
Original, mean	83.34	85.33	+1.99
Alternate, seed 42	84.39	85.71	+1.32
Anonymized queries			
Original, seed 42	72.74	62.71	-10.03
Original, seed 1	51.42	61.91	+10.49
Original, seed 2	66.98	50.21	-16.77
Original, mean	63.71	58.28	-5.44
Alternate, seed 42	63.75	72.82	+9.06
Fine-tune on half A			
Split / seed	No PT	PT on B	Δ
Original queries			
Original, seed 42	80.57	93.65	+13.08
Original, seed 1	80.05	91.16	+11.11
Original, seed 2	79.93	85.87	+5.94
Original, mean	80.18	90.23	+10.04
Alternate, seed 42	78.66	85.00	+6.34
Anonymized queries			
Original, seed 42	54.69	58.00	+3.31
Original, seed 1	63.96	49.64	-14.32
Original, seed 2	64.39	48.04	-16.35
Original, mean	61.01	51.89	-9.12
Alternate, seed 42	52.25	52.19	-0.07

B. Ablation Study

Table IV examines pooling, similarity scoring, passage length, pre-training objectives, and pre-training duration. Results use a single random seed for both original and anonymized queries, with separate pre-training on each source corpus.

a) Embedding Construction: On original queries, first-token configurations average 17.01–17.87. Switching to mean pooling raises the average to 87.48 with dot-product scoring and 92.65 with cosine similarity. For mean pooling with cosine similarity, raising the passage length from 256 to 1024 tokens increases the average from 55.97 to 92.65. The same increase changes the CodeT5 first-token average by only 0.55 points. On anonymized queries, mean pooling with cosine similarity similarly rises from 29.99 to 86.49. These findings indicate that XML retrieval therefore depends on both how the embedding is built and how much of the document the model sees.

b) Pre-training Objectives and Duration: Without pre-training, CodeT5 scores 91.67 on original queries and 89.68 on anonymized queries. SDA alone averages 88.62 and 82.72, MEP alone averages 91.10 and 83.47, and the combined objectives average 92.65 and 86.49. Adding MEP to SDA does not reduce the reported score for any source in either query set. Adding SDA to MEP is less consistent. On anonymized queries, it lowers the BIKE and FOSSIG scores by 5.59, 0.79, but raises the s1kd-tools score by 15.48 points.

Extending pre-training from 10 to 100 epochs lowers the average from 92.65 to 89.15 on original queries and 86.49 to 77.17 on anonymized queries. BIKE and FOSSIG drop the most on anonymized queries (14.96 and 11.83 points). Compared with no pre-training, the combined pre-training tasks at 10 epochs improve the original-query average by 0.98 points but reduce the anonymized-query average by 3.19 points. At 100 epochs, both averages fall below the no-pre-training baseline. Therefore, more pre-training does not consistently improve retrieval in these experiments.

c) Same-Domain Pre-training: We split the 297 S1000D spec training documents into halves A (150) and B (147). One half is used for SDA/MEP pre-training and the other for fine-tuning; the comparison model is fine-tuned on the same half without pre-training. Table V reports three seeds and an alternate, independently hashed split of 143/154 documents. For original queries, pre-training raises the mean from 83.34 to 85.33 when fine-tuning on half B, and from 80.18 to 90.23 when fine-tuning on half A. For anonymized queries, however, the mean changes are -5.44 and -9.12 points. Effects also vary by seed, and the alternate split gives anonymized-query changes of +9.06 and -0.07 points.

Without pre-training, models on the original split score 79.93 – 86.31 on original queries and 51.42–72.74 on anonymized queries. With pre-training, the anonymized-query

changes on this split range from -16.77 to +10.49 points across seeds.

C. Embedding Visualization of Structured and Unstructured Data

Figure 2 displays the 21 XML document embeddings and 21 query embeddings with t-SNE. Circles denote documents, triangles denote queries, and lines connect matched pairs. These plots provide a qualitative view of the embedding space, as line lengths and distances in a t-SNE projection do not directly measure similarity in the original space. Table VI therefore reports cosine similarities computed from the original embeddings. Doc–doc and query–query average distinct pairs; query–doc averages matched pairs, while query–other averages each query against the remaining 20 documents. All pre-trained models in this comparison use BIKE.

First-token embeddings form two compact groups, separating queries from documents (Fig. 2(a)). Mean document–document cosine similarity rounds to 1.000, indicating nearly identical directions, while query–query similarity is 0.985. Each query has the same average similarity to its relevant document and to other documents (both 0.127 at the reported precision). This lack of separation is consistent with the model's low retrieval score of 15.38.

Mean pooling produces more dispersed representations (Fig. 2(b)–(f)). With dot-product scoring, document–document similarity is 0.632, and the mean similarity gap between matched and unmatched query–document pairs is 0.269. Query–query similarity decreases from 0.985 for first-token pooling to 0.588 for mean pooling with dot-product scoring and to 0.351–0.385 for the mean-pooling cosine configurations.

With cosine scoring, document–document similarity is 0.412 without pre-training and 0.445–0.497 with pre-training. The matched-versus-unmatched similarity gap is 0.383 without pre-training, 0.319 with SDA only, 0.385 with MEP only, and 0.367 with both tasks. The model without pre-training has the highest MRR among these BIKE comparisons (91.67), even though MEP alone has a slightly larger average similarity gap. Average similarity separation therefore does not by itself determine retrieval ranking. Queries and documents remain largely in separate regions of the mean-pooled projections. These observations describe representation behavior, but do not establish that specific XML tags, attributes, or cross-node relations have been preserved.

D. Case Studies

Table VII compares CodeT5 pre-trained with SDA/MEP on FOSSIG against CodeT5 without pre-training. For the table-of-contents query, pre-training moves the relevant document from rank 4 to rank 1. Without pre-training, the highest-ranked document concerns building SCORM content package modules. For the controls-and-indicators and page-oriented-publications queries, both models retrieve the relevant document first.

These examples illustrate a case where pre-training improves ranking and two cases where both models succeed. However, the original queries contain topic phrases and chapter information that overlap with document titles. Successful retrieval may therefore reflect lexical and semantic matching as well as any structural information in the representations.

TABLE VI
MEAN EMBEDDING COSINE SIMILARITIES (CODET5, ORIGINAL QUERIES; BIKE PRE-TRAINING). D: DOCUMENT; Q: QUERY. GAP IS Q–D MINUS Q–OTHER; ONLY MRR IS SCALED BY 100.

D–D	Q–Q	Q–D	Q–Other	Gap	MRR
First-token, dot (+ SDA/MEP)					
1.000	0.985	0.127	0.127	0.000	15.38
Mean, dot (+ SDA/MEP)					
0.632	0.588	0.701	0.432	0.269	85.60
Mean, cosine, no pre-training					
0.412	0.368	0.568	0.185	0.383	91.67
Mean, cosine (+ SDA only)					
0.497	0.382	0.507	0.188	0.319	83.33
Mean, cosine (+ MEP only)					
0.445	0.351	0.598	0.213	0.385	88.78
Mean, cosine (+ SDA/MEP)					
0.497	0.385	0.639	0.272	0.367	89.84

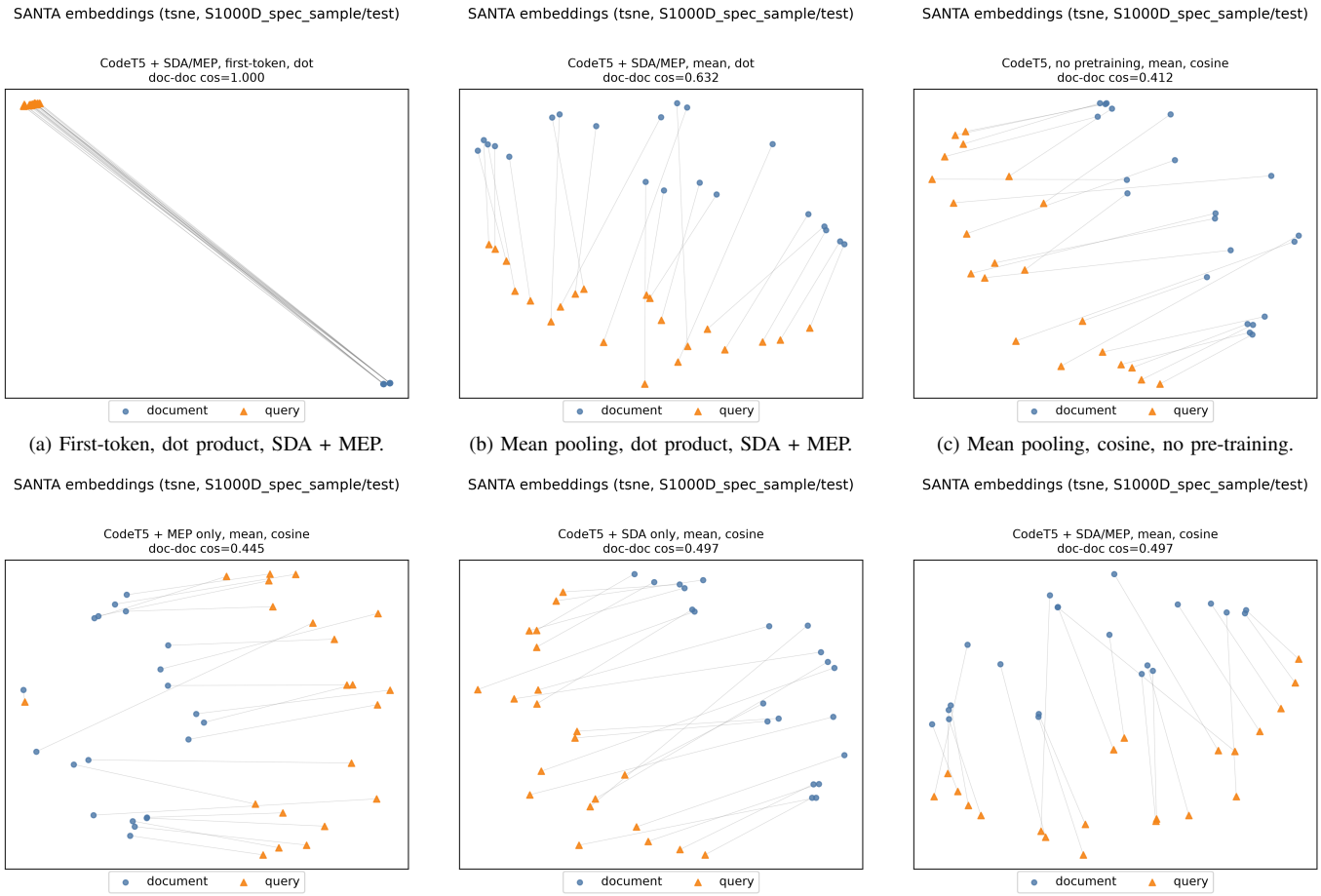
TABLE VII
RETRIEVAL CASES WITH ORIGINAL QUERIES AND 21 CANDIDATES. RANK REFERS TO THE RELEVANT DOCUMENT; ⋆ MARKS A CORRECT TOP 1.

Model	CodeT5 + SDA/MEP (FOSSIG)	CodeT5, no pre-training
Query	This S1000D technical document is a front matter table of contents module, issued December 31, 2016 (issue 008) by the Aerospace and Defence Industries Association of Europe …	
Rank	1	4
Top 1	00-00-0000 Table of contents ⋆	04-15-0300 Learning information - Building SCORM content package modules
Query	This S1000D technical document is a common information repository module for aircraft controls and indicators (Chapter 3.9.5.2.11.11) …	
Rank	1	1
Top 1	03-09-0502 Common information repository - Controls and indicators ⋆	03-09-0502 Common information repository - Controls and indicators ⋆
Query	This S1000D technical documentation module specifies Chapter 6.2 on information presentation and use for page-oriented publications …	
Rank	1	1
Top 1	06-02-0000 Information presentation and use - Page-oriented publications ⋆	06-02-0000 Information presentation and use - Page-oriented publications ⋆

VI. CONCLUSIONS

This work evaluated SANTA for S1000D XML document retrieval and examined the effects of representation choices, pre-training settings, and retrieval conditions. Mean-pooled encoder outputs with cosine similarity significantly improved retrieval over representations from the first decoding step, while longer document inputs also improved performance. A possible reason is that a single decoder state captures little of a long, nested data module, whereas mean pooling could more fully represent every encoded token.

The benefits of SDA and MEP pre-training varied with the data source, training duration, and query formulation, with no consistent advantage. BM25 performed well with a small



(a) First-token, dot product, SDA + MEP.

(b) Mean pooling, dot product, SDA + MEP.

(c) Mean pooling, cosine, no pre-training.

(d) Mean pooling, cosine, MEP only.

(e) Mean pooling, cosine, SDA only.

(f) Mean pooling, cosine, SDA + MEP.

Fig. 2. t-SNE of 21 documents (circles) and matched queries (triangles). Lines connect relevant pairs. CodeT5 is used throughout, with BIKE for pre-training.

candidate pool, but dense retrieval performed better when the pool was expanded. Conclusions about these models depend on the evaluation setting as much as on the model itself.

The work is limited by the small number of test queries, and retrieval performance alone does not show that the model preserves specific XML relationships. Concluded with the anonymized-query, candidate-pool, and case-study results, this work shows which retrieval configurations work and where they may fall short. We see establishing this as a necessary step before investigating structure-preserving generation. Future work will expand the document and query sets, test how changes to tags, attributes, and hierarchy affect retrieval, and explore the use of these representations for S1000D document generation.

References

[1] ASD, AIA, and A4A, "S1000D, international specification for technical publications using a common source database," n.d. [Online]. Available: https://s1000d.org/

[2] S. Zhang, "A method for the automatic generation of technical manuals for civil aircraft wiring lists," in *2024 4th International Conference on Electronic Information Engineering and Computer Communication (EIECC)*. IEEE, 2024, pp. 1112–1118.

[3] K. Cui, Q. Jiang, Y. Zhou, and Z. Jia, "Research on the preparation method of aircraft system wiring diagram manual based on S1000D," in *Mechanical and Aerospace Engineering*. IOS Press, 2026, pp. 170–179.

[4] T. El Khoury, G. Badr, A. H. El Hassani, and S. N'Guyen Van Ky, "S1000d AppLicability dEpended pRocessiNg mOdel – SALERNO," in *2023 14th International Conference on Information, Intelligence, Systems & Applications (IISA)*. IEEE, 2023, pp. 1–7.

[5] T. El Khoury, G. Badr, A. H. El Hassani, and S. N. Van Ky, "Applicability filtering model for S1000D documents using AND/OR trees," in *2024 15th International Conference on Information, Intelligence, Systems & Applications (IISA)*. IEEE, 2024, pp. 1–4.

[6] X. Li, Z. Liu, C. Xiong, S. Yu, Y. Gu, Z. Liu, and G. Yu, "Structure-aware language model pretraining improves dense retrieval on structured data," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 11 560–11 574.

[7] A.-M. Vercoustre, M. Fegas, S. Gul, and Y. Lechevallier, "A flexible structured-based representation for XML document mining," in *Advances in XML Information Retrieval and Evaluation*. Springer, 2006, pp. 443–457.

[8] K. S. Tai, R. Socher, and C. D. Manning, "Improved semantic representations from tree-structured long short-term memory networks," in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, 2015, pp. 1556–1566.

[9] W. Woof and K. Chen, "A framework for end-to-end learning on semantic tree-structured data," *arXiv preprint arXiv:2002.05707*, 2020.

[10] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *The Semantic Web*. Springer, 2018, pp. 593–607.

[11] J. Li, Y. Xu, L. Cui, and F. Wei, "MarkupLM: Pre-training of text and markup language for visually rich document understanding," in *Proceedings of the 60th Annual Meeting of the Association for Computational*

Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, 2022, pp. 6078–6087.

[12] X. Deng, P. Shiralkar, C. Lockard, B. Huang, and H. Sun, "DOM-LM: Learning generalizable representations for HTML documents," *arXiv preprint arXiv:2201.10608*, 2022.

[13] Q. Wang, Y. Fang, A. Ravula, F. Feng, X. Quan, and D. Liu, "WebFormer: The web-page transformer for structure information extraction," in *Proceedings of the ACM Web Conference 2022*. Association for Computing Machinery, 2022, pp. 3124–3133.

[14] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[15] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.

[16] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, G. Li, L. Zhou, L. Shou, L. Zhou, M. Tufano, M. Gong, M. Zhou, N. Duan, N. Sundaresan, S. K. Deng, S. Fu, and S. Liu, "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021.

[17] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, 2020, pp. 1536–1547.